

Kai Yi Ng, Assignment 1

Repo Link:

Question 1

Run: time python3 matmul_slow.py

```
ubuntu@ubuntu:~/CWM-project/assignment1$ time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
.
.
.
real    0m0.232s
user    0m0.224s
sys     0m0.008s
```

Real time: total time

User time: time CPU spent running program code

Sys time: time CPU spent doing OS tasks (e.g. loading data into memory)

Real = user + sys

Run: /usr/bin/time -v time python3 matmul_slow.py

```

ubuntu@ubuntu:~/CWM-project/assignment1$ /usr/bin/time -v time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
0.21user 0.01system 0:00.23elapsed 99%CPU (0avgtext+0avgdata 13428maxresident)k
702inputs+0outputs (0major+1947minor)pagefaults 0swaps
  Command being timed: "time python3 matmul_slow.py"
  User time (seconds): 0.21
  System time (seconds): 0.01
  Percent of CPU this job got: 98%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.23
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 13428
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 2035
  Voluntary context switches: 45
  Involuntary context switches: 8
  Swaps: 0
  File system inputs: 702
  File system outputs: 0
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
ubuntu@ubuntu:~/CWM-project/assignment1$ 

```

Gives more information about why the program is slow

Question 2

```

ubuntu@ubuntu:~/CWM-project/assignment1$ for i in 1 2 3 4 5; do /usr/bin/time -f "%e" python3 matmul_slow.py; done
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22

```

The mean will be affected by anomalies in the run.

Median is better than mean in that it will not be pulled up/ down by anomalies.

However, since the goal is to characterize the program (ideal), we would want the best-case time. Hence, we should report the minimum.

Question 3

```
ubuntu@ubuntu:~/CWM-project/assignment1$ sudo perf stat -e cycles,instructions python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py':

      807279234      cycles
    2299621064      instructions          #    2.85  insn per cycle

0.231382048 seconds time elapsed

0.221441000 seconds user
0.008977000 seconds sys
```

Question 4

```
ubuntu@ubuntu:~/CWM-project/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py' (5 runs):

      811869215      cycles                               ( +- 0.29% )
    2295103994      instructions                #    2.83  insn per cycle       ( +- 0.04% )
      840735        cache-misses                               ( +- 5.03% )

0.227340 +- 0.000669 seconds time elapsed ( +- 0.29% )
```

2.83 instructions per cycle. This can be a measure of efficiency. Low instruction per cycle means CPU does less work in one cycle.

840735 cache misses. High cache misses mean that accessed data is not stored readily in the cache, which means it takes longer to retrieve data from the slower RAM. Hence, the program is slower.

Question 5

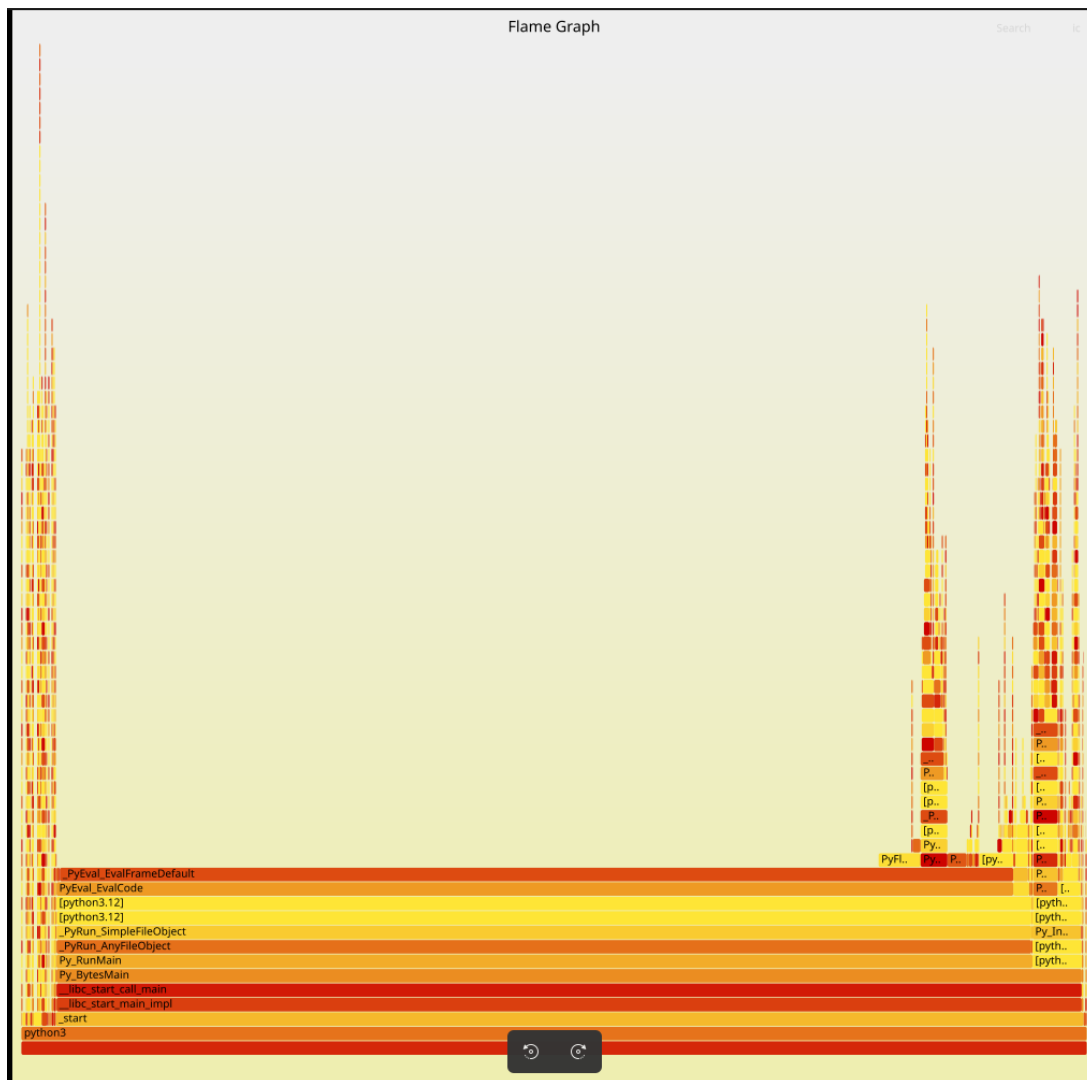
PyevalFrameDefault consumes the largest overhead. We should look at the “self” column as that ignores the children.

ubuntu@ubuntu: ~/CWM-project/assignment1

Samples: 920 of event 'cycles:P', Event count (approx.): 807879514

	Children	Self	Command	Shared Object	Symbol
+	99.81%	0.00%	python3	python3.12	[.] _start
+	99.81%	0.00%	python3	libc.so.6	[.] __libc_start_main@@GLIBC_2.34
+	99.81%	0.00%	python3	libc.so.6	[.] __libc_start_call_main
+	99.81%	0.00%	python3	python3.12	[.] Py_BytesMain
+	95.56%	0.00%	python3	python3.12	[.] PyEval_EvalCode
+	93.54%	74.30%	python3	python3.12	[.] _PyEval_EvalFrameDefault
+	92.84%	0.00%	python3	python3.12	[.] Py_RunMain
+	92.84%	0.00%	python3	python3.12	[.] _PyRun_AnyFileObject
+	92.84%	0.00%	python3	python3.12	[.] _PyRun_SimpleFileObject
+	90.75%	0.00%	python3	python3.12	[.] 0x000000000006b5263
+	90.53%	0.00%	python3	python3.12	[.] 0x00000000000608b52
+	8.37%	0.00%	python3	python3.12	[.] PyImport_ImportModuleLevelObject
+	8.37%	0.00%	python3	python3.12	[.] PyObject_CallMethodObjArgs
+	8.37%	0.00%	python3	python3.12	[.] 0x0000000000054a087
+	7.94%	0.00%	python3	python3.12	[.] 0x0000000000058206d
+	7.62%	0.00%	python3	python3.12	[.] 0x000000000005d36ac
+	6.97%	0.00%	python3	python3.12	[.] 0x000000000006bc965
+	6.85%	0.00%	python3	python3.12	[.] 0x000000000006bca35
+	6.85%	0.00%	python3	python3.12	[.] Py_InitializeFromConfig
+	5.65%	0.00%	python3	python3.12	[.] 0x000000000006b0bc4
+	5.57%	0.00%	python3	python3.12	[.] 0x000000000005d39f4
+	5.24%	0.00%	python3	python3.12	[.] PyObject_CallFunction
+	5.03%	0.00%	python3	python3.12	[.] PyImport_ImportModule
+	5.03%	0.00%	python3	python3.12	[.] PyImport_Import
+	4.79%	0.00%	python3	python3.12	[.] PyObject_Vectorcall
+	4.40%	3.52%	python3	python3.12	[.] PyFloat_FromDouble
+	4.01%	0.00%	python3	python3.12	[.] 0x000000000006b1132
+	3.74%	3.08%	python3	python3.12	[.] PyLong_FromLong
+	2.96%	0.10%	python3	python3.12	[.] _PyObject_MakeTpCall
+	2.57%	0.00%	python3	python3.12	[.] 0x000000000005d3759
+	2.57%	0.00%	python3	python3.12	[.] PyRun_StringFlags
+	2.22%	0.00%	python3	python3.12	[.] 0x00000000000599c17
+	1.87%	0.00%	python3	python3.12	[.] 0x000000000006b542c

Question 6



PyEvalframe is the dominating function as it is the one in which afterwards the children all become very narrow.

This is the evaluation frame. Having a long run time suggests a loop.

Looking at the code it corresponds to the triple nested loop. The code does not exploit spatial locality, which means it has to retrieve from the slower RAM.

```
# Intentionally simple O(n^3) matrix multiplication.
# This loop order is correct but cache-unfriendly for matrix B.
def matmul_slow(a: Matrix, b: Matrix, c: Matrix, n: int) -> None:
    for i in range(n):
        for j in range(n):
            total = 0.0
            for k in range(n):
                total += a[i][k] * b[k][j]
            c[i][j] = total
```

|

Question 7

We should try to optimize the evaluation phase since that dominates run time.

Looking at `matmul_fast`, it uses three different optimisations. Matrices are stored in memory row by row so to make use of the cache we would want to use row access instead of column access.

First is reordering loops to improve locality:

It saves each row `a[i]` and `c[i]` as a separate variable outside of the inner loops. This allows us to only index into `rowai[k]` inside the `k` loop instead of having to look up `a[i]` in every iteration of the `k` loop. Now we only look up `a[i]` once and save it to `row_ai` during each `i` loop.

Second is reordering to reduce inner loops

We exploit spatial locality because now the inner-most loop accesses `row_bk` sequentially. This means that we can exploit spatial locality because we are accessing each data from left to right and hence cache misses will be reduced.

Also, `aik` is saved as a scalar. No need for repeated lookups in each iteration of the loop.

Third is the matrix transpose method

The row of `bt[j]` is now the column of `b`. So, by doing so we can exploit row access as we are now accessing row of `bt` instead. This reduces cache misses (hence faster program)

Looking at matmul_fast.py

Fast1:

Basic timing:

```
real    0m0.213s
user    0m0.201s
sys     0m0.010s
```

Verbose:

```
n=128 reps=2 checksum=107389.290000
  Command being timed: "python3 matmul_fast.py"
  User time (seconds): 0.20
  System time (seconds): 0.00
  Percent of CPU this job got: 100%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.21
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 13344
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 1963
  Voluntary context switches: 1
  Involuntary context switches: 1
  Swaps: 0
  File system inputs: 0
  File system outputs: 0
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
```

Repeat 5 runs

```

n=128 reps=2 checksum=107389.290000
0.21
n=128 reps=2 checksum=107389.290000
0.21
n=128 reps=2 checksum=107389.290000
0.21
n=128 reps=2 checksum=107389.290000
0.21
n=128 reps=2 checksum=107389.290000
0.21
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

```

Perf stats:

```

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

```

```

      756414157      cycles
( +- 0.49% )
    2060096597      instructions          #    2.72  insn per cycle
( +- 0.04% )
      1380599      cache-misses          #   11.91% of all cache refs
( +- 6.57% )
     11588933      cache-references
( +- 1.31% )

```

```

    0.21216 +- 0.00124 seconds time elapsed ( +- 0.59% )

```

```

n=128 reps=2 checksum=107389.290000

```

Flame graph:


```
real    0m0.212s
user    0m0.201s
sys      0m0.010s
n=128 reps=2 checksum=107389.290000
  Command being timed: "python3 matmul_fast.py"
  User time (seconds): 0.19
  System time (seconds): 0.00
  Percent of CPU this job got: 100%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.20
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 13300
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 1964
  Voluntary context switches: 1
  Involuntary context switches: 1
  Swaps: 0
  File system inputs: 0
  File system outputs: 0
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
```

```
n=128 reps=2 checksum=107389.290000
0.20
n=128 reps=2 checksum=107389.290000
0.20
n=128 reps=2 checksum=107389.290000
0.20
n=128 reps=2 checksum=107389.290000
0.20
n=128 reps=2 checksum=107389.290000
0.20
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
```

Performance counter stats for 'python3 matmul_fast.py' (5 runs):

```
747350486      cycles
( +- 0.50% )
2060155824      instructions          #    2.76  insn per cycle
( +- 0.04% )
819335          cache-misses          #    6.58% of all cache refs
( +- 9.89% )
12447606         cache-references
( +- 4.39% )

0.20992 +- 0.00116 seconds time elapsed ( +- 0.55% )
```

n=128 reps=2 checksum=107389.290000

[perf record: Woken up 1 times to write data]

[perf record: Captured and wrote 0.053 MB perf.data (891 samples)]

Fast 3:

```
real    0m0.213s
user    0m0.205s
sys      0m0.007s
n=128 reps=2 checksum=107389.290000
Command being timed: "python3 matmul_fast.py"
User time (seconds): 0.19
System time (seconds): 0.00
Percent of CPU this job got: 99%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.20
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
```

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